Project Description

Secure Data Exchange System

The goal of this project is to **design and implement a secure data exchange system** that guarantees the **confidentiality, integrity, and authenticity** of transmitted messages or files through a **hybrid cryptographic architecture**.

The proposed system must integrate **symmetric encryption**, **public-key cryptography**, and **digital signatures** to ensure secure communication, user authentication, and message integrity verification.

System Components

The solution should incorporate the following elements:

- A **symmetric encryption algorithm** implemented in a selected mode of operation
- A **public-key (asymmetric) cryptographic algorithm**
- A **digital signature scheme** for authentication and integrity verification
- A **verified user authentication mechanism**
- Implementation in any suitable programming language (e.g., Python, MATLAB, C++, Java)

The use of **high-level cryptographic libraries** (e.g., import cryptography in Python or #include cryptographic toolkits in C/C++) is **not permitted**. However, external support may be used **only for hash functions and key generation**.

A graphical user interface (GUI) is **optional**; however, the system must provide a precise mechanism for **input/output control and evaluation**.

Project Deliverables

Students are required to submit the following:

- A **one-page report (in English)** summarizing the project methodology, system flow, and experimental results
- The complete **source code**
- A **PowerPoint presentation (in English)** describing the project

Presentation and Evaluation

- The project will be presented **in class** in a **20-minute oral presentation**, followed by **10 minutes of questions** on both the project and the relevant course material
- Students must explain not only the implementation details but also the **theoretical background**, including:
  - The motivation for selecting the chosen cryptographic techniques
  - An evaluation and interpretation of the results obtained

Implementation Notes

- Architectures such as “**client–server”** are not necessary.